

COS 214 Practical 2

Student Numbers:

- u25138309 – Christian Nunes
- u24742512 – Thabo Ngwabi

Question 1

1.1)

	Trying to be	Should be
Movement:	Strategy	State
Routes:	State	Strategy
Map:	Composite	Composite
Place Features:	Decorator	Decorator
Biomes:	Abstract Factory	Abstract Factory

Routes and Movement are swapped. Their design patterns need to be swapped back as Movement is better represented by state that which the player is in and Routes need to choose between different ways to calculate the route; the intents match better with the design patterns.

1.2) State(Movement): Context: Traveller, State: TravelState, ConcreteState: Eg:

WalkingState, DrivingState, SwimmingState, FlyingState, etc

Strategy (Routes): Context: Trip, Strategy: RouteStrategy, Concrete Strategy: Eg:

FastestRoute, ShortestRoute, ScenicRoute, etc.

Composite(Map): Client: Not a class (something abstract like code),

Component: MapComponent, Leaf: Location, Composite: Region

Decorator(Place Features): Component: MapComponent, ConcreteComponent:

Location, Decorator: Decorator, ConcreteDecorator: Eg:

WhetherDecorator, TerrainDecorator, etc.

Abstract Factory(Biomes): AbstractFactory: WorldFactory, ConcreteFactory:

DesertFactory, VolcanoFactory, etc. AbstractProduct: AbstractNPC,

AbstractTerrain, AbstractObstacle, ConcreteProduct: DesertTerrain,

DesertNPC, etc. Client: MapComponent or abstract (game code).

1.3) Wrong Pattern Structure/Choice: Movement uses Strategy (The pattern's intent is being violated and as such movement becomes harder to model due to its inability to represent the traveller's state change.), Routes use State (The pattern's intent is reversed with Strategy and as such it misrepresents the problem and thus makes the design less flexible and harder to maintain.), Decorator is implemented using subclasses for combinations (It is required that there is no subclass-per-combination as it would defeat the whole purpose of Decorator. Thus, as consequence the number of classes grows rapidly as more features are added. This leads to very difficult maintenance.)

Wrong UML Structure: No abstract classes (The patterns used require the use of abstract classes for interfaces and thus there is no proper abstraction being used when defining interfaces and concrete implementations.)

Incorrect Composite Structure (Location and Region don't properly share a common Component interface due to the use of members in Map. This breaks the Composite structure and thus we can't treat Region and Location uniformly through a component interface.)

Broken Ownership: Objects are not deleted (It is stated that nothing is ever deleted. This allows for Memory leaks due to the owner not being able to free what it owns),

Missing Virtual Destructors (Base classes that rely on polymorphic use need virtual destructors and as such if their derived classes cannot call their destructors when they destruct could lead to memory leaks or undefined behaviour.)

Excessive Coupling: GameManager has excessive Coupling (It is responsible for every subsystem and thus is excessively coupled. This causes the responsibilities of every pattern to be centralised into one central class causing the class to become difficult to maintain and use as if one subsystem changes the GameManager too must change.)

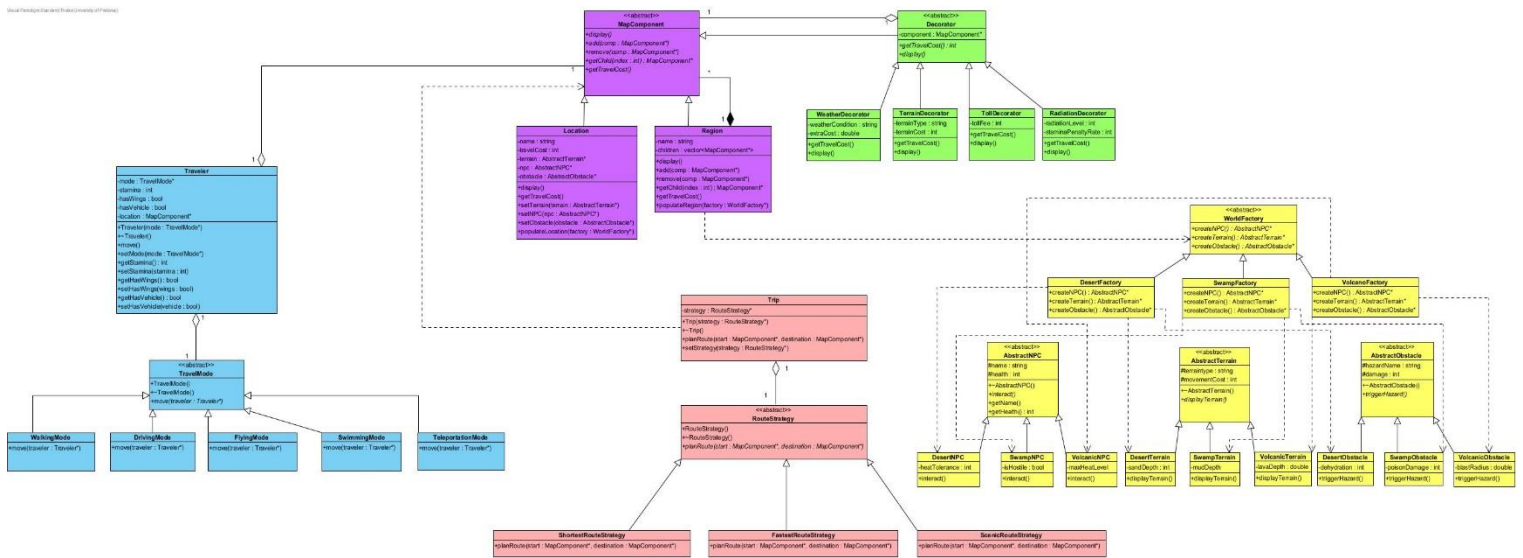
1.4) GameManager knows about everything. This breaks the purpose of design patterns to separate the responsibilities of every subsystem. The GameManager class should coordinate the systems rather than implement all the behaviour. By centralizing all the behaviour, if one subsystem is changed, GameManager needs to change itself as well, defeating coupling goals. Traveller should own its TravelState. Trip should own its RouteStrategy. Region should own its MapComponents. Decorator should own the MapComponent it wraps. WorldFactory (and its concrete implementations) should own the Products they create.

1.5) The diagram is hard to read because too many classes, relationships, and pattern labels are crowded together or overlapping, making the structure unclear.

It could be improved by separating each subsystem, grouping related classes, using consistent spacing, and clearly showing the relationships between them.

Question 2

2.1)



2.2)

Composite:

- Component: MapComponent
- Composite: Region
- Leaf: Location

Decorator:

- Component: MapComponent
- ConcreteComponent: Location
- Decorator: Decorator
- ConcreteDecorator: WeatherDecorator, TerrainDecorator, TollDecorator

Strategy:

- Context: Trip
- Strategy: RouteStrategy
- ConcreteStrategies: ShortestRouteStrategy, FastestRouteStrategy, ScenicRouteStrategy

AbstractFactory

- AbstractFactory: WorldFactory
- ConcreteFactories: DesertFactory, SwampFactory
- AbstractProducts: AbstractNPC, AbstractTerrain, AbstractObstacle

State:

- Context: Traveler
- State: TravelMode
- ConcreteStates: WalkingMode, DrivingMode, FlyingMode, SwimmingMode

Abstraction/Ownership Design Justification

- Region and MapComponent, a composition association was used because when one region is deleted, all it's other regions/locations down the tree hierarchy chain must be deleted as well

2.3)

Addition 1: A new biome (Abstract Factory)

- Created VolcanoFactory which creates VolcanicNPC, VolcanicTerrain, VolcanicObstacle

Addition 2: A new environment decorator (Decorator)

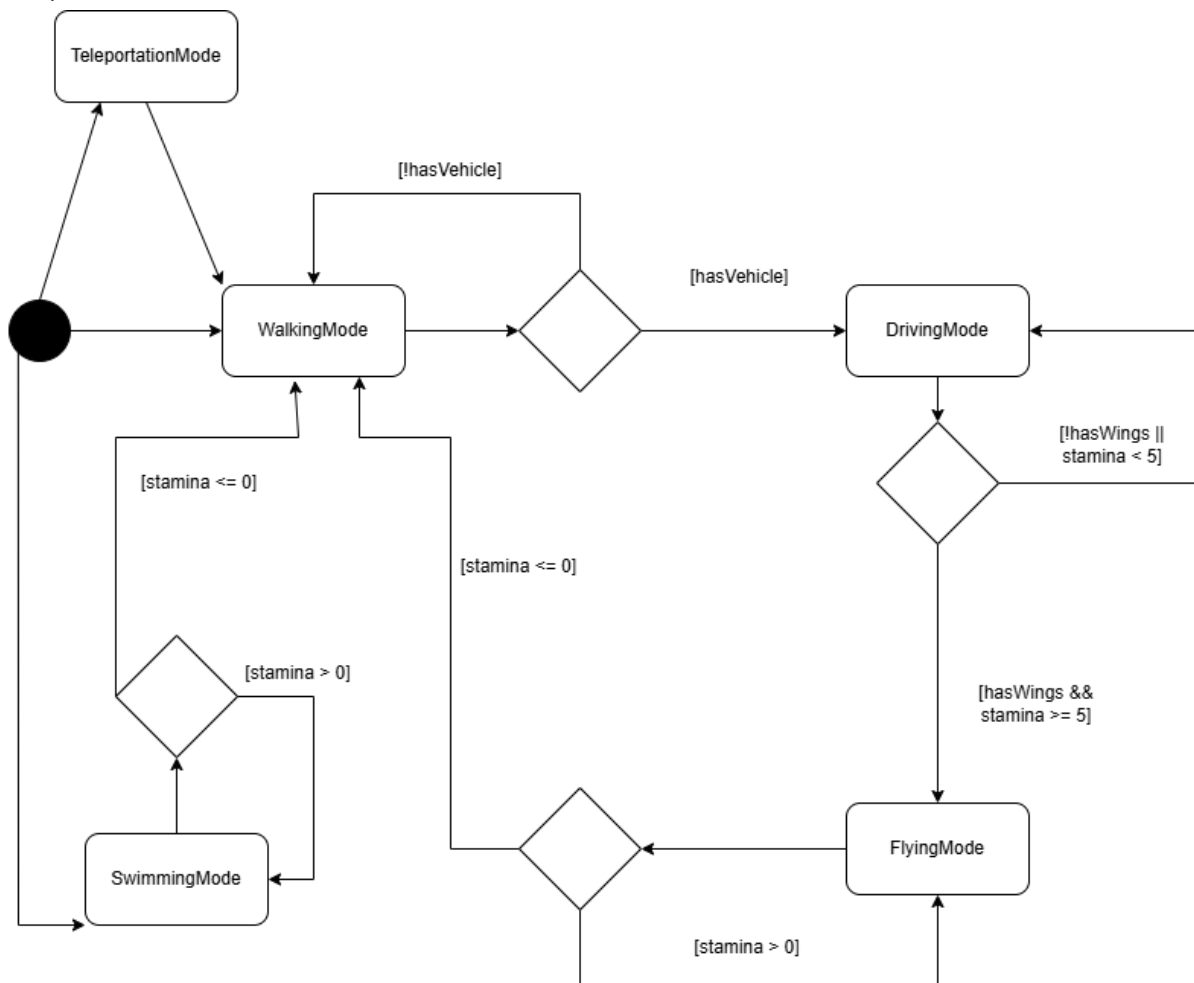
- Created RadiationDecorator which would add massive travel cost penalties

Addition 3: TeleportationMode

- Added a fast travel teleportation mode

Question 3

3.3)



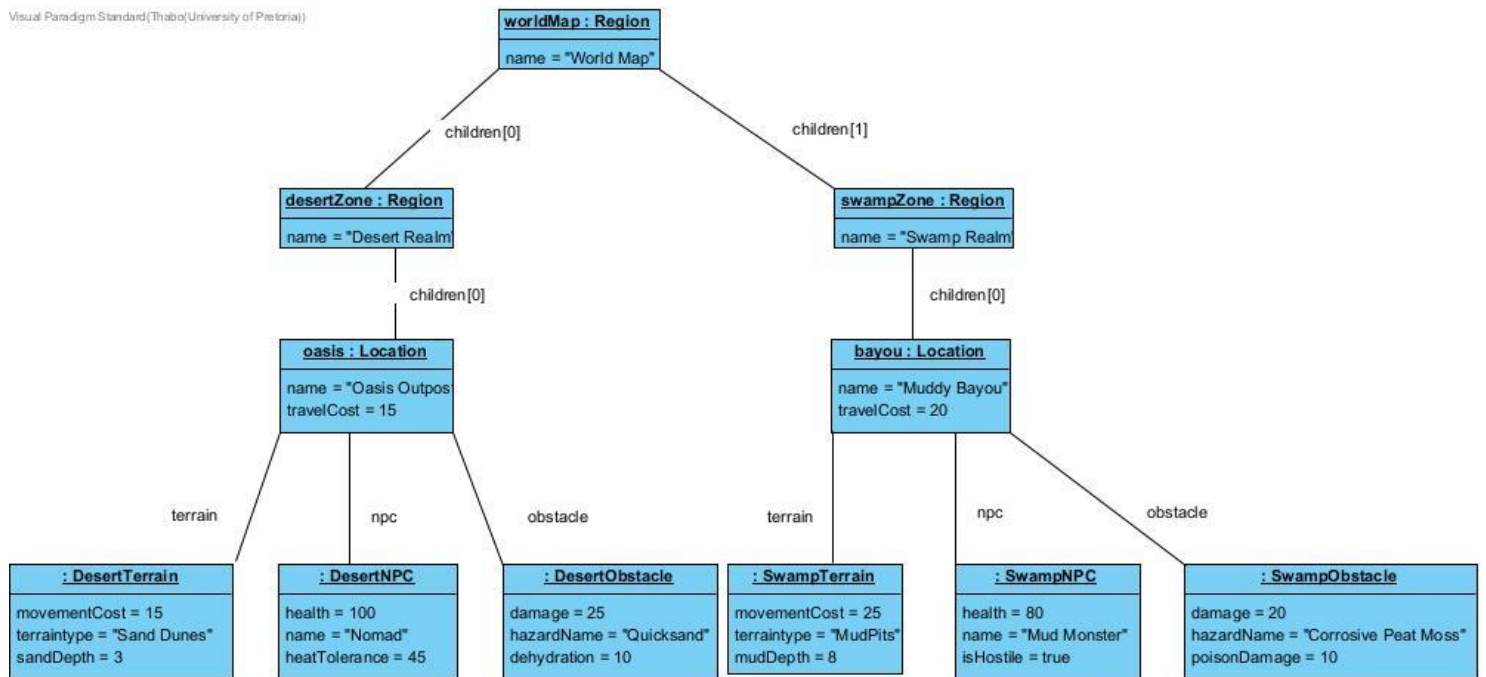
Question 4

4.3) The intent of Strategy is to define a family of algorithms, encapsulate them and then make them interchangeable, allowing clients that use it to have varying algorithms. The intent of State is to allow an object to alter its behaviour based off of its internal state, as if it were changing its class. Strategy is used for routes because Trip needs varying route-finding methods that should be independent of Trip. State is used for movement because Traveller changes its behaviour based off of its current mode (of which is also changed in correlation to the internal state of Traveller). Strategy chooses the algorithm manually while State changes behaviour based on the object's current internal state.

Question 5

5.2)

Visual Paradigm Standard (Thabo University of Pretoria)



Question 6

6.2)

